GNU Octave

Proposal - Google Summer of Code 2026

Add `anova` Classdef Object to the GNU Octave Statistics Package

Contact

- Name: Aman Behera
- Email: aman.behera.systesms@gmail.com
- Octave discourse: https://octave.discourse.group/u/beingamanforever/
- Github: https://github.com/beingamanforever
- LinkedIn: https://www.linkedin.com/in/aman-behera-/
- City: Roorkee
- Country: India
- Timezone: IST (Indian Standard Time) (UTC +5:30)
- Language: English

Abstract

The GNU Octave statistics package currently provides ANOVA functionality through three separate procedural functions — `anova1`, `anova2`, and `anovan`, each with its own calling convention, output format, and stats structure. Users who want to fit a model, inspect results, run post-hoc tests, and generate diagnostic plots must juggle multiple function calls and manually pass structs between them.

This project implements an `anova` classdef that provides a unified, stateful, object-oriented interface for analysis of variance. The class delegates all core computation to the existing `anova1`, `anova2`, and `anovan` functions, preserving their tested numerical logic while wrapping it in a clean API with properties (coefficients, ANOVA table, residuals, fitted values, diagnostics) and methods (`summary`, `multcompare`, `plotDiagnostics`, `predict`, `getEffectSizes`). Users interact with one object instead of coordinating three functions and their disparate output structs.

As additional value, the project delivers targeted performance improvements to the underlying procedural functions — vectorizing loop-heavy computations in `anova1` and `anova2`, reducing memory overhead in `anovan`'s interaction column construction, and adding NaN-tolerant handling to `anova2`, so the classdef wrapper sits on top of faster, more robust internals.

Problem Statement

1. No OOP Interface for ANOVA

MATLAB provides class-based statistical modeling objects (e.g., `LinearModel` from `fitlm` with an `anova` method). Octave's statistics package has no equivalent. Users must:

```
Python
% Current workflow — manual, stateless, error-prone
[p, atab, stats] = anovan(Y, {g1, g2}, 'model', 'full', 'display', 'off');
% Now manually pass stats to multcompare
c = multcompare(stats);
% Want diagnostics? Manually extract residuals from stats struct
residuals = stats.resid;
% Want to refit with different SS type? Call anovan again from scratch
[p2, atab2, stats2] = anovan(Y, {g1, g2}, 'model', 'full', 'sstype', 2,
'display', 'off');
```

The desired workflow:

```
Python
a = anova(Y, {g1, g2}, 'model', 'full');
```

```
a.summary()              % formatted ANOVA table
a.multcompare('g1')      % post-hoc for factor 1
a.plotDiagnostics()      % residual plots
a.SSType = 2;            % change SS type, lazy refit
a.summary()              % updated table
```

2. Inconsistent Stats Structs Across Functions

- `anova1` returns `stats.means`, `stats.gnames`, `stats.n`, `stats.df`, `stats.s`
- `anova2` returns `stats.sigmasq`, `stats.colmeans`, `stats.rowmeans`, `stats.pval`, `stats.model`
- `anovan` returns `stats.coeffs`, `stats.resid`, `stats.X`, `stats.vcov`, `stats.mse`, etc.

A classdef unifies these into consistent, documented properties.

3. Performance Bottlenecks in Underlying Functions

While building the classdef, I will also address concrete performance issues in the functions it delegates to:

- `anova1` (lines 119–125): Per-group stats via `for j = 1:groups` with `find(group_id == j)` each iteration: O(N×G) instead of O(N)
- `anova2` (lines 98–103): Within-cell SSE via double nested loop — O(FFGn × SFGn × reps) with repeated indexing
- `anova2` (line 38): Hard error on NaN instead of handling missing data
- `anovan` (lines ~560): Interaction columns via `num2cell`/`cell2mat` round-trip — unnecessary intermediate allocations

Proposed Architecture

Class Design: `anova.m` (classdef)

Python

```
classdef anova < handle

  properties (SetAccess = private)
    % Data
    Y               % Response vector
    GROUP           % Grouping variables (cell array)

    % Results (populated after fit)
    Coefficients    % Model coefficients table
    AnovaTable      % ANOVA table (cell array, same format as atab)
```

```
  Residuals       % Weighted residuals
  FittedValues    % Predicted values from the model
  DFE             % Residual degrees of freedom
  MSE             % Mean squared error
  DesignMatrix    % Sparse design matrix
  Stats           % Full stats struct (for multcompare compatibility)
end

properties (Access = public)
  % Model specification (changing these triggers lazy refit)
  ModelType       % 'linear', 'interaction', 'full', or terms matrix
  SSType          % 1, 2, or 3
  VarNames        % Factor names
  Contrasts       % Contrast coding specification
  Alpha           % Confidence level
  Continuous      % Indices of continuous predictors
  Random          % Indices of random factors
  Weights         % Observation weights
  Display         % 'on' or 'off'
end

properties (Access = private)
  fitted_         % Flag: has the model been fit?
  dirty_          % Flag: do properties need refit?
  nFactors_       % Number of factors
  backend_        % Which function was used: 'anova1','anova2','anovan'
end

methods (Access = public)
  % Constructor
  function obj = anova(Y, GROUP, varargin)
  % summary - display formatted ANOVA table
  function summary(obj)
  % multcompare - post-hoc pairwise comparisons
  function [c, m, h, gnames] = multcompare(obj, varargin)
  % plotDiagnostics - residual, QQ, leverage, Cook's D plots
  function plotDiagnostics(obj)
  % predict - fitted values for new data
  function yhat = predict(obj, newGROUP)
  % getEffectSizes - eta-squared, partial eta-squared, omega-squared
  function es = getEffectSizes(obj)
  % disp - custom display for command window
  function disp(obj)
end
```

```
  methods (Access = private)
    % fit_ - delegates to anova1/anova2/anovan based on data shape
    function fit_(obj)
    % ensureFit_ - lazy refit if dirty
    function ensureFit_(obj)
    % selectBackend_ - choose anova1/anova2/anovan
    function selectBackend_(obj)
  end
end
```

How Delegation Works

The class does not reimplement SS computation, design matrices, or QR solves. The private `fit_()` method selects and calls the appropriate existing function:

Python

```
function fit_(obj)
    switch obj.backend_
        case 'anova1'
            [p, atab, stats] = anova1(obj.Y, obj.GROUP{1}, 'off', ...);
        case 'anova2'
            [p, atab, stats] = anova2(obj.Y, obj.reps_, 'off', ...);
        case 'anovan'
            [p, atab, stats] = anovan(obj.Y, obj.GROUP, ...
                'model', obj.ModelType, 'sstype', obj.SSType, ...
                'varnames', obj.VarNames, 'contrasts', obj.Contrasts, ...
                'continuous', obj.Continuous, 'random', obj.Random, ...
                'weights', obj.Weights, 'alpha', obj.Alpha, ...
                'display', 'off');
    endswitch
    % Populate properties from stats struct
    obj.Coefficients = stats.coeffs;
    obj.Residuals = stats.resid;
    obj.DFE = stats.dfe;
    obj.MSE = stats.mse;
    obj.DesignMatrix = stats.X;
    obj.AnovaTable = atab;
    obj.Stats = stats;
    obj.FittedValues = full(stats.X) * stats.coeffs(:,1);
    obj.fitted_ = true;
```

```
    obj.dirty_ = false;
endfunction
```

Backend Selection Logic

```Python
function selectBackend_(obj)
    if obj.nFactors_ == 1 && isempty(obj.Continuous) && ...
       isempty(obj.Weights) && obj.SSType == 3
        obj.backend_ = 'anova1';
    elseif obj.nFactors_ == 2 && isempty(obj.Continuous) && ...
           ismatrix(obj.Y) && ~any(isnan(obj.Y(:))) && ...
           isempty(obj.Weights)
        obj.backend_ = 'anova2';
    else
        obj.backend_ = 'anovan';
    endif
endfunction
```

This means:

- Simple one-way problems use the fast `anova1` path
- Balanced two-way problems use `anova2` (exact arithmetic, no QR)
- Everything else goes through `anovan` (full generality)

Lazy Refit Mechanism

When a user changes a model specification property (e.g., `obj.SSType = 2`), a `set` method marks the object as dirty. The next time any result property is accessed or a method is called, `ensureFit_()` checks the flag and refits if needed:

```Python
function ensureFit_(obj)
    if ~obj.fitted_ || obj.dirty_
        obj.selectBackend_();
        obj.fit_();
    endif
endfunction
```

Methods in Detail

`summary(obj)`

- Calls `ensureFit_()`, then formats and prints the ANOVA table. Reuses the existing print formatting from `anovan` (lines 440–480) but extracted into a shared helper to avoid duplication. Also prints the model formula, effect sizes (partial eta-squared), and significance codes.

`multcompare(obj, varargin)`

- Delegates directly to the existing `multcompare.m` function using the stored `obj.Stats`:

```python
function [c, m, h, gnames] = multcompare(obj, varargin)
    obj.ensureFit_();
    [c, m, h, gnames] = multcompare(obj.Stats, varargin{:});
endfunction
```

No reimplementation of multcompare — just pass-through with the correct stats struct. This ensures the classdef immediately benefits from any future improvements to `multcompare.m`.

`plotDiagnostics(obj)`

- Generates the same four diagnostic plots currently in `anovan` (lines 486–540): Normal Q-Q, Spread-Location, Residual-Leverage, Cook's Distance. The plotting code will be extracted from `anovan` into a private helper `__anova_plot_diagnostics__` that both `anovan` and the classdef can call, eliminating the ~55 lines of duplicated plotting logic.

`predict(obj, newGROUP)`

- Constructs the design matrix for new observations using the stored contrast matrices and computes `X_new * obj.Coefficients(:,1)`. This reuses the contrast functions already in `anovan` (`contr_simple`, `contr_poly`, etc.).

`getEffectSizes(obj)`

- Computes and returns a struct with:
  - Eta-squared: SS_effect / SS_total (already computed in `anovan`, stored in `stats.eta_squared`)
  - Partial eta-squared: SS_effect / (SS_effect + SS_error) (already in `stats.partial_eta_squared`)
  - Omega-squared: (SS_effect - df_effect × MSE) / (SS_total + MSE) — new, computed from stored ANOVA table values
  - No new linear algebra: just arithmetic on values already in the ANOVA table.

Performance Improvements to Underlying Functions

These are independent, self-contained patches that make the classdef's backend faster. Each is separately mergeable.

Change 1: Vectorize `anova1` group statistics

Current code (`anova1.m`, lines 119–125):

```
Python
for j = 1:groups
    group_size = find(group_id == j);
    xs(j) = length(group_size);
    xm(j) = mean(xr(group_size));
    xv(j) = var(xr(group_size), 0);
endfor
```

Proposed replacement:

```
Python
xs = accumarray(group_id, 1, [groups, 1])';
xm = accumarray(group_id, xr, [groups, 1], @mean)';
xv = accumarray(group_id, xr, [groups, 1], @(v) var(v, 0))';
```

This eliminates the per-group `find()` scan (each O(N)), reducing total complexity from O(N×G) to O(N). This is the same pattern I used in my `silhouette` and `pdist` blocked computation patches.

Change 2: Vectorize `anova2` within-cell SSE

Current code (`anova2.m`, lines 98–103):

```
Python
SSE = 0;
for i = 1:FFGn
    for j = 1:SFGn
        SSE += sum((x(RIdx(i,:),j) - mean(x(RIdx(i,:),j))) .^ 2);
    endfor
```

```
endfor
```

Proposed replacement using the algebraic identity `SS_within = sum(x²) - sum(cell_sum² / cell_n)`:

```python
## Reshape x into (reps × FFGn × SFGn) and compute cell-level SS in one pass
xr = reshape(x, reps, FFGn, SFGn);
cell_means = mean(xr, 1);
SSE = sum((xr - cell_means).^2, "all");
```

This eliminates the double loop entirely. For a 100×50 design with 10 reps, this avoids 5000 loop iterations.

Change 3: Add NaN handling to `anova2`

Currently `anova2` fails hard on NaN. The fix replaces the NaN error with row-wise exclusion (listwise deletion) for the balanced case, and per-cell exclusion for unbalanced analysis. This will be guarded by a check that the design remains balanced after exclusion; if not, a warning directs users to `anovan`.

The change touches the input validation section (lines 36–40) and the group statistics computation. The existing balanced-design formulas remain unchanged — only the data fed into them changes.

Change 4: Reduce memory in `anovan` interaction column construction

In `mDesignMatrix` (anovan.m, ~line 560), interaction columns are built by:

```python
for j = 1:numel(I)
    tmp = num2cell(tmp, 1);
    for k = 1:numel(tmp)
        tmp(k) = bsxfun(@times, tmp{k}, X{I(j)});
    endfor
    tmp = cell2mat(tmp);
endfor
```

This creates intermediate cell arrays and materializes the full Kronecker product. Instead, I'll compute interaction columns directly using element-wise products of the factor columns, avoiding the cell-array intermediate allocation:

```
Python

tmp = X{I(1)};

for j = 2:numel(I)

    cols_a = size(tmp, 2);

    cols_b = size(X{I(j)}, 2);

    tmp = reshape(tmp, n, cols_a, 1) .* reshape(X{I(j)}, n, 1, cols_b);

    tmp = reshape(tmp, n, cols_a * cols_b);

endfor
```

This avoids the `num2cell` / `cell2mat` round-trip and reduces peak memory by eliminating intermediate cell arrays. For a 3-factor interaction with factors of 5, 4, 3 levels (producing 60 columns from 4×3×2 = 24 contrast columns), this avoids allocating and converting 24 separate cell entries.

Change 5: Improve `multcompare` integration

The `stats` struct returned by `anova2` is missing fields that `multcompare` expects for certain model types (e.g., nested model doesn't populate `inter` correctly when `reps > 1` is forced to 1). I'll audit the `stats` struct population in `anova1`, `anova2`, and `anovan` for consistency with what `multcompare` actually reads, and add missing fields or correct types.

Backward Compatibility

- Existing functions unchanged at the API level: `anova1`, `anova2`, `anovan` retain identical signatures and return values
- Performance patches are internal: Same outputs, faster computation
- All existing BISTs pass: The 3 test blocks for `anova1`, 3 for `anova2`, and 12 for `anovan` serve as regression tests
- The classdef is purely additive: New file `@anova/anova.m` (or `inst/anova.m` depending on package convention), no modification to existing class hierarchy
- `multcompare` delegation: The classdef uses the existing `multcompare` function, not a reimplementation

Expected Impact

Metric	Before	After (expected)
User workflow	3 separate functions, manual struct passing	Single `anova` object with methods
Post-hoc tests	`multcompare(stats)` must keep stats variable	`a.multcompare()`: stats stored in object
Diagnostics	Only in `anovan`, inline plotting code	`a.plotDiagnostics()` from any ANOVA type
Effect sizes	Partial eta-sq only in `anovan`	Eta-sq, partial eta-sq, omega-sq via `a.getEffectSizes()`
`anova1` (100 groups, N=100k)	O(N×G) loop-based	O(N) accumarray-based
`anova2` SSE (50×30, 10 reps)	1500 loop iterations	Single vectorized operation
`anova2` NaN support	Hard error	Listwise deletion with balanced check
`anovan` interaction memory	Cell-array intermediates	Direct reshape product

All improvements are backward-compatible: same inputs produce same outputs (within floating-point tolerance), verified by existing BISTs.

Deliverables

1. `anova` classdef — constructor, properties, backend selection, lazy refit
2. `summary` method — formatted ANOVA table with effect sizes and significance codes
3. `multcompare` method — pass-through to existing **`multcompare.m`**
4. **`plotDiagnostics`** method — shared diagnostic plots (extracted from **`anovan`**)
5. `predict` method — fitted values for new data using stored design matrix
6. `getEffectSizes` method — eta-squared, partial eta-squared, omega-squared
7. `disp` method — clean command-window display of model summary
8. Performance patches — vectorized **`anova1`**, **vectorized `anova2` SSE, NaN handling, memory-optimized `anovan` interactions**
9. `multcompare` stats struct fixes — consistent fields across all backends
10. BISTs — comprehensive tests for classdef + all performance patches
11. Texinfo documentation — full docstring for classdef with examples
12. Benchmark suite — timing/memory measurements before and after optimizations

Project Schedule (12 Weeks)

Weeks (Timeline)	Tasks assigned
Pre-GSoC	Study Octave classdef system (syntax, handle vs value classes, property access). Read existing classdef examples in statistics package (e.g., `fitcdiscr`, `ClassificationKNN`). Profile anova1/anova2/anovan on large datasets. Study the related issue in the statistics repo.
Community Bonding Period (May 15 - May 28)	Finalize class API with mentor (property names, method signatures, backend selection heuristics). Confirm classdef file placement convention (`@anova/` vs `inst/`). Set up a benchmark harness.
Week 1 (Jun 1 - Jun 7)	Implement classdef skeleton: constructor, property declarations, input validation, `selectBackend_()`. Parse name-value pairs in constructor. Add basic BISTs for construction.
Week 2 (Jun 8 - June 14)	Implement `fit_()` and `ensureFit_()`. Wire delegation to `anova1`/`anova2`/`anovan`. Populate all result properties from stats structs. Test with one-way, two-way balanced, and N-way data.
Week 3 (Jun 15 - Jun 21)	Implement `summary()` and `disp()`. Extract ANOVA table formatting into shared helper. Format output with effect sizes, significance codes, model formula. BISTs for display output.
Week 4 (Jun 22 - Jun 28)	Implement `multcompare()` method. Fix stats struct inconsistencies in `anova2` for nested/linear models so delegation works for all backends. BISTs covering multcompare from all three backends.
Week 5 (Jun 29 - July 5)	Implement `plotDiagnostics()`. Extract plotting code from `anovan` into `__anova_plot_diagnostics__` private function shared by both `anovan` and the classdef. BISTs (test figure creation, no visual regression).
Week 6 (July 6 - July 12)	Implement `predict()` and `getEffectSizes()`. For predict: construct design matrix for new observations using stored contrasts. For effect sizes: compute omega-squared from ANOVA table. BISTs. Submit classdef for review.
Midterm evaluation	Complete classdef with all methods. All BISTs passing.
Week 7 (July 13 - July 19)	Performance patch 1: vectorize `anova1` group stats. Performance patch 2: vectorize `anova2` SSE. Run all existing BISTs. Benchmark before/after.

Week 8 (July 20 - July 26)	Performance patch 3: NaN handling in `anova2`. Performance patch 4: memory-optimized `anovan` interaction columns. BISTs for NaN edge cases. Memory benchmarks.
Week 9 (July 27 - Aug 2)	Address mentor review feedback on classdef and performance patches. Edge-case hardening: single-group, single-observation, all-identical values, empty groups, 100+ factor levels. Lazy refit correctness tests.
Week 10 (Aug 3 - Aug 9)	Implement lazy refit tests (change SSType, ModelType, Contrasts — verify results update correctly). Test classdef with all existing `anovan` demo datasets. Cross-validate results against procedural function calls.
Week 11 (Aug 10 - Aug 16)	Texinfo documentation for all public methods and properties. Add demo blocks showing full workflows (fit → summary → multcompare → plot). Coding standards compliance review.
Week 12 (Aug 17 - Aug 25)	Final benchmark suite. Clean up all patches for independent mergeability. Final mentor review. Ensure all deliverables are complete and documented.
Final evaluation	
Post-GSoC	Maintain patches through merge. Address post-merge bug reports. Explore extending classdef: ANCOVA workflows, repeated measures support, integration with future `fitlm`/`LinearModel` class.

Experience & Readiness

Contributions to GNU Octave (Statistics Package):

Pull Request	Status
Kdtree optimize query performance with inline distance computation (#384)	Merged
shape bug fixed, added BISTs (#383)	Merged
squareform: drop symmetry warning for MATLAB compatibility (#379)	Merged
Fix rangesearch kdtree guards (#378)	Merged
blocked pdist.m, memory optimized to O(n·p) (#370)	Merged
Optimize ridge.m (#374)	Merged
silhouette: avoid full distance matrix, use blocked computation (#372)	Merged
crosstab: add categorical vector input support (#351)	Merged
pdist2: loop optimization (#371)	Merged
rnd: accept empty size vector as third argument (#367)	Merged
Lower memory footprint of knnsearch, rangesearch using block processing (#361)	Merged
ExhaustiveSearcher: optimize pdist2 call order, fix IncludeTies logic (#360)	Merged
tabulate: fix categorical and string edge cases (#352)	Merged
hnswSearcher: batch best-list sort per iteration (#358)	Merged
hnswSearcher: optimize search layer with preallocation + batch pdist2 (#357)	Merged

Contributions to GNU Octave (Core Repository upstream)

Pull Request	Status
median: use `nth_element` for linear-time selection on dense inputs (#40)	Merged

Some other proposals by me as discussed in the community:

1. Exploring simdjson as an alternative backend for jsonencode/jsondecode
2. Fundamental performance limits of pure Octave HNSW & possible C++ backend

Updated merges can be seen here -
https://github.com/gnu-octave/statistics/commits?author=beingamanforever

Bio

Hi, I am Aman, a 4th year B.Tech student from IIT Roorkee, one of India's topmost technical institutions. I work primarily around C++ infra, ML systems and anything related to performance. My interests lie in algorithmic efficiency, memory behaviour, and reducing unnecessary abstractions. I like to contribute to open source softwares, majority of my contributions have been to gnu-octave (statistics package, core octave) and RISCV, Earth-imagery ML models.I also hold a Pre Placement offer from HiLabs.Inc as a Data Scientist

I am a part of ACM (Association of Computer Machinery) and VLG (Vision and Language group) student-run technical groups that aim to foster Machine Learning and Core computing culture on campus, which gave me insights to computer core and machine learning advancements.

My key notable projects include: Drishti (Scalable remote sensing pipeline for unified satellite imagery analysis), zeroshred (P2P file transfer system in C++), CSV parsing engine (Single thread CSV parser optimized for numeric data, built with SWAR), working on Generative engine optimization as an Intern at Infoedge India Ltd.

Every contribution follows the same pattern: identify a measurable issue, apply a targeted fix, preserve the API, add BISTs. This is exactly the methodology for both the classdef implementation (building on existing functions) and the performance patches (optimizing existing code paths) :)